

Autodesk® Scaleform®

Scaleform 4.2

Scaleform 4.2

: Michael Antonov
: 1.03
: 8.2.2012

Copyright Notice

Autodesk® Scaleform® 4.2

© 2012 Autodesk, Inc. All rights reserved. Except as otherwise permitted by Autodesk, Inc., this publication, or parts thereof, may not be reproduced in any form, by any method, for any purpose.

Certain materials included in this publication are reprinted with the permission of the copyright holder.

The following are registered trademarks or trademarks of Autodesk, Inc., and/or its subsidiaries and/or affiliates in the USA and other countries: 123D, 3ds Max, Algor, Alias, AliasStudio, ATC, AUGI, AutoCAD, AutoCAD Learning Assistance, AutoCAD LT, AutoCAD Simulator, AutoCAD SQL Extension, AutoCAD SQL Interface, Autodesk, Autodesk Homestyler, Autodesk Intent, Autodesk Inventor, Autodesk MapGuide, Autodesk Streamline, AutoLISP, AutoSketch, AutoSnap, AutoTrack, Backburner, Backdraft, Beast, Beast (design/logo) Built with ObjectARX (design/logo), Burn, Buzzsaw, CAiCE, CFdesign, Civil 3D, Cleaner, Cleaner Central, ClearScale, Colour Warper, Combustion, Communication Specification, Constructware, Content Explorer, Creative Bridge, Dancing Baby (image), DesignCenter, Design Doctor, Designer's Toolkit, DesignKids, DesignProf, DesignServer, DesignStudio, Design Web Format, Discreet, DWF, DWG, DWG (design/logo), DWG Extreme, DWG TrueConvert, DWG TrueView, DWFx, DXF, Ecotect, Evolver, Exposure, Extending the Design Team, Face Robot, FBX, Fempro, Fire, Flame, Flare, Flint, FMDesktop, Freewheel, GDX Driver, Green Building Studio, Heads-up Design, Heidi, Homestyler, HumanIK, i-drop, ImageModeler, iMOUT, Incinerator, Inferno, Instructables, Instructables (stylized robot design/logo), Inventor, Inventor LT, Kynapse, Kynogon, LandXplorer, Lustre, MatchMover, Maya, Mechanical Desktop, MIMI, Moldflow, Moldflow Plastics Advisers, Moldflow Plastics Insight, Moondust, MotionBuilder, Movimento, MPA, MPA (design/logo), MPI (design/logo), MPX, MPX (design/logo), Mudbox, Multi-Master Editing, Navisworks, ObjectARX, ObjectDBX, Opticore, Pipeplus, Pixlr, Pixlr-o-matic, PolarSnap, Powered with Autodesk Technology, Productstream, ProMaterials, RasterDWG, RealDWG, Real-time Roto, Recognize, Render Queue, Retimer, Reveal, Revit, RiverCAD, Robot, Scaleform, Scaleform GFx, Showcase, Show Me, ShowMotion, SketchBook, Smoke, Softimage, Sparks, SteeringWheels, Stitcher, Stone, StormNET, Tinkerbox, ToolClip, Topobase, Toxik, TrustedDWG, T-Splines, U-Vis, ViewCube, Visual, Visual LISP, Vtour, WaterNetworks, Wire, Wiretap, WiretapCentral, XSI.

All other brand names, product names or trademarks belong to their respective holders.

Disclaimer

THIS PUBLICATION AND THE INFORMATION CONTAINED HEREIN IS MADE AVAILABLE BY AUTODESK, INC. "AS IS." AUTODESK, INC. DISCLAIMS ALL WARRANTIES, EITHER EXPRESS OR

IMPLIED, INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE REGARDING THESE MATERIALS.

How to Contact Autodesk Scaleform:

Document	Scaleform 4.2 Renderer Threading Guide
Address	Scaleform Corporation 6305 Ivy Lane, Suite 310 Greenbelt, MD 20770, USA
Website	www.scaleform.com
Email	info@scaleform.com
Direct	(301) 446-3200
Fax	(301) 446-3199

차례

1		1
2		1
2.1		2
2.2		3
2.3		4
2.4		5
2.5		7
3		8
3.1		9
4	:	10
5		12
5.1		12
5.2		12
5.3		15
6	GfxShaderMaker	16
6.1		16
6.2	API HAL.....	16
6.2.1	D3D9	16
6.2.2	D3D1x.....	17

1 소개

Scaleform 4.0 ()
Scaleform SDK
Scaleform 4.0 API
Scaleform

2 멀티 쓰레딩 렌더러 변경 내용

- 4.0 API Scaleform 3.x Scaleform
1. Renderer2D
 2. GFxLoader
 3. - (thread-safe)
 4. MovieDisplayHandle
Renderer2D::Display
 5. GFx::Movie
 6. API
Scaleform 3.X GRenderer Render::HAL

2.1

```
Scaleform                                Render::Renderer2D    Render::HAL
      . Renderer2D                                     Display
      . Render::HAL    Renderer2D                    “
”
,              Render::D3D9::HAL    Render::PS3::HAL
```

```
D3DPRESENT_PARAMETERS    PresentParams;
IDirect3DDevice9*         pDevice = ...;
ThreadId                  renderThreadId = Scaleform::GetCurrentThreadId();
Render::ThreadCommandQueue *commandQueue = ...;
Ptr<Render::D3D9::HAL>    pHal;
Ptr<Render::Renderer2D>  pRenderer;

pHal = *SF_NEW Render::D3D9::HAL(commandQueue);

if (!pHal->InitHAL(Render::D3D9::HALInitParams(pDevice, PresentParams, 0,
                                                renderThreadId)))
{
    return false;
}
pRenderer = *SF_NEW Render::Renderer2D(pHal);
```

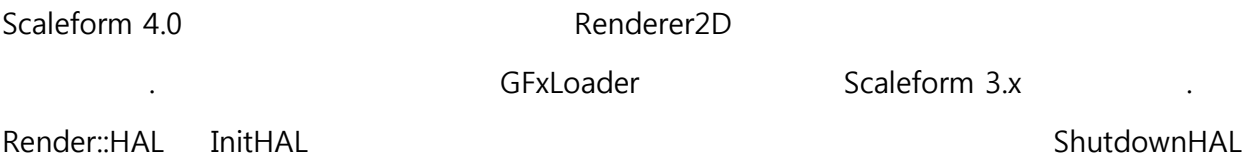
```

    , HALInitParams
    pDevice    PresentParams
renderThreadId . ThreadCommandQueue

```

Renderer2D HAL - (thread-safe)

2.2



Render::HAL

HAL

```
MeshCacheParams mcp;
mcp.MemReserve      = 1024 * 1024 * 3;
mcp.MemLimit        = 1024 * 1024 * 12;
mcp.MemGranularity  = 1024 * 1024 * 3;
mcp.LRUTailSize     = 1024 * 1024 * 4;
mcp.StagingBufferSize = 1024 * 1024 * 2;

pRenderer->GetMeshCacheConfig()->SetParams(mcp);
```

, MemReserve MemLimit



PC

```
GlyphCacheParams gcp;
gcp.TextureWidth  = 1024;
gcp.TextureHeight = 1024;
gcp.NumTextures   = 1;
gcp.MaxSlotHeight = 48;
```

```
pRenderer->GetGlyphCacheConfig()->SetParams(gcp);
```

```

Scaleform 3.3
TextureWidth, TextureHeight
MaxSlotHeight
InitHAL
SetParams

```

2.3

```
Scaleform 4.0
Render::TextureManager
ImageCreator
Gfx::Loader
Gfx::ImageCreator
Scaleform,
```

```
Gfx::Loader loader;
...

SF::Ptr<Gfx::ImageCreator> pimageCreator =
    *new Gfx::ImageCreator(pRenderThread->GetTextureManager());
loader.SetImageCreator(pimageCreator);
```

```

graph TD
    ImageCreator --> TextureManager
    TextureManager --> Render
    Render --> TextureManager
    TextureManager --> TextureManager

```

Render::HAL , CreateTexture TextureManager
 - (thread-safe) .
 .
 • D3DCREATE_MULTITHREADED
 D3D9 - (thread-safe) .
 • ThreadCommandQueue ,
 Render::HAL ..
D3D
ThreadCommandQueue .
 2 . to
 Platform::RenderHALThread .

2.4

. advance
 advance/ "draw frame" -
 . Scaleform 4.0 APIs Gfx::MovieDisplayHandle
 . Gfx::Movie -
 Display ..
 .
 1. . Gfx::Movie Gfx::MovieDef::CreateInstance
 , .
 2. . Advance
 ActionScript . Advance
 Invoke/DirectAccess API .
 . Advance Scaleform
 .

3.

MovieDisplayHandle

```
//-----  
// 1.  
  
Ptr<Gfx::Movie>      pMovie = ...;  
Gfx::MovieDisplayHandle hMovieDisplay;  
  
//  
//  
pMovie->SetViewport(width, height, 0,0, width, height);  
hMovieDisplay = pMovie->GetDisplayHandle();  
  
//                                     . Scaleform Player  
//  
pRenderThread->AddDisplayHandle(hMovieDisplay);  
  
//-----  
// 2.  
  
//  
// ExternalInterface  
float deltaT = ...;  
pMovie->HandleEvent(...);  
pMovie->Advance(deltaT);  
  
//  
//  
pRenderThread->WaitForOutstandingDrawFrame();  
pRenderThread->DrawFrame();  
  
//-----  
// 3.      -      DrawFrame  
Ptr<Render::Renderer2D> pRenderer = ...;  
  
pRenderer->BeginFrame();  
bool hasData = hMovieDisplay.NextCapture(pRenderer->GetContextNotify());  
if (hasData)  
    pRenderer->Display(hMovieDisplay);  
pRenderer->EndFrame();
```

- WaitForOutstandingFrame – CPU
 . Advance
 .
- MovieDisplay.NextCapture – Advance " "
 .
 false

2.5

GFx::Movie Release .

Render::ThreadCommandQueue Movie .

ThreadCommandQueue .

, advance

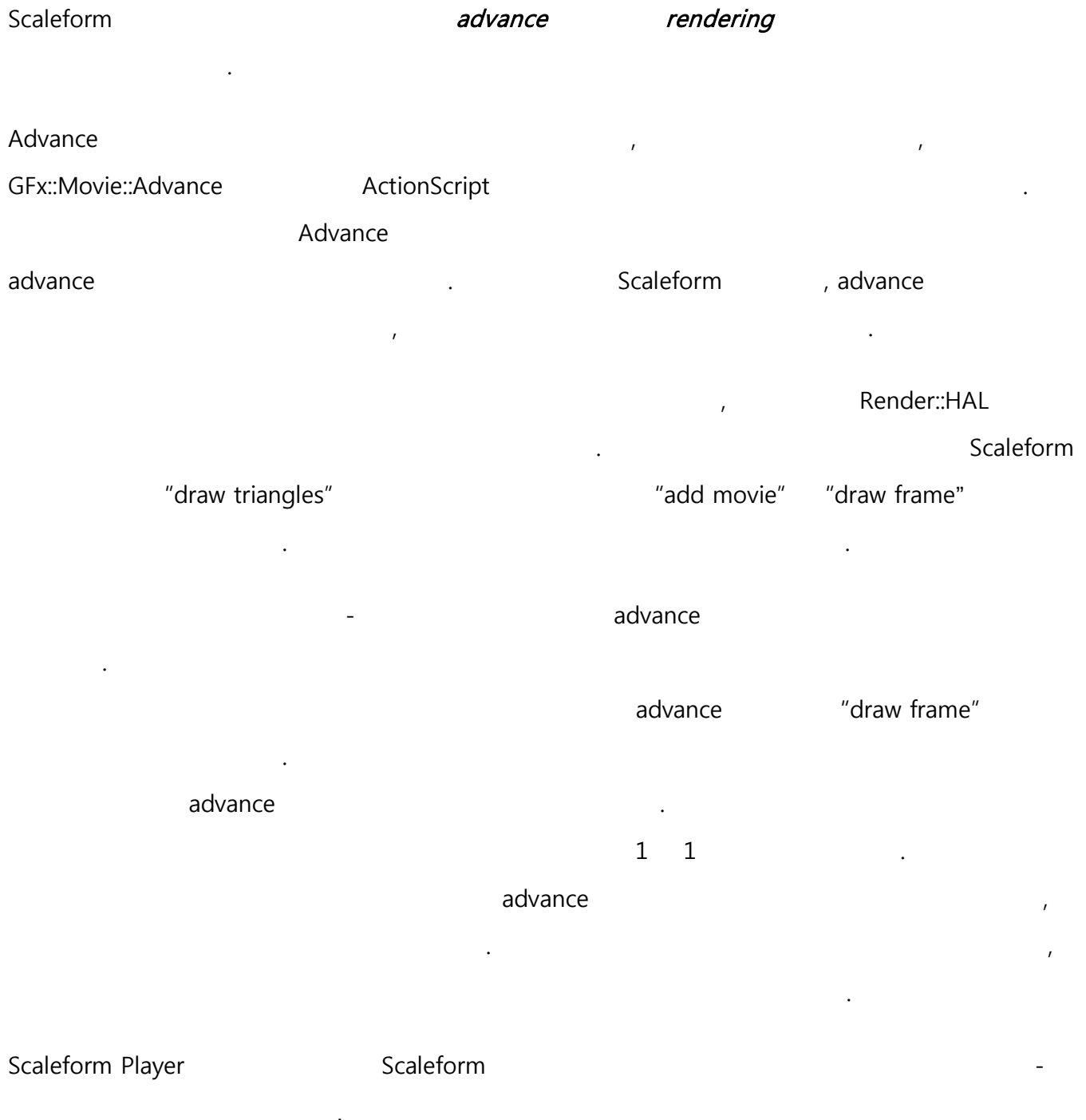
Movie::ShutdownRendering(false)

Movie::IsShutdownRenderingComplete .

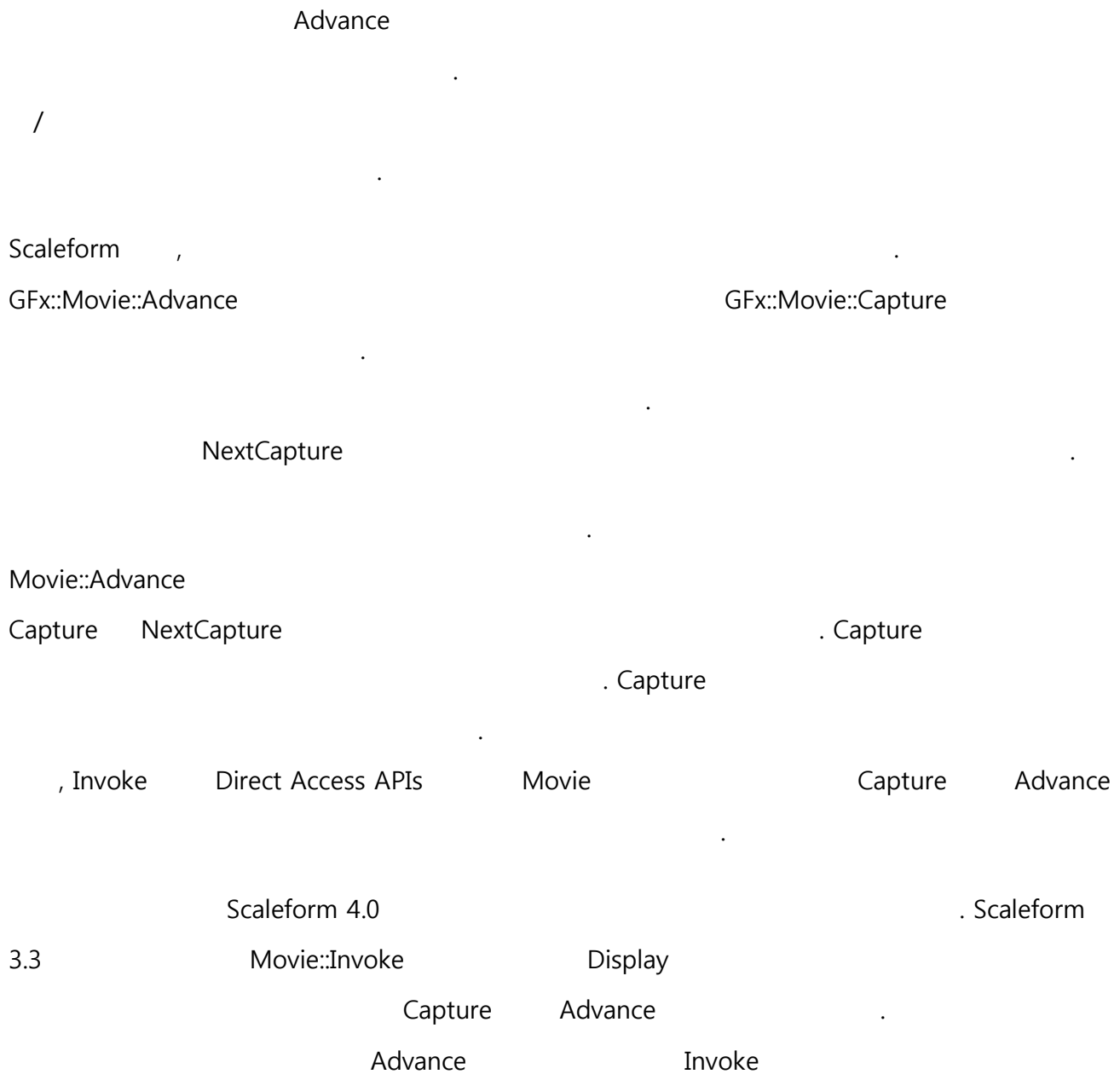
, NextCapture false . advance

.

3 멀티 쓰레드 렌더링 개념

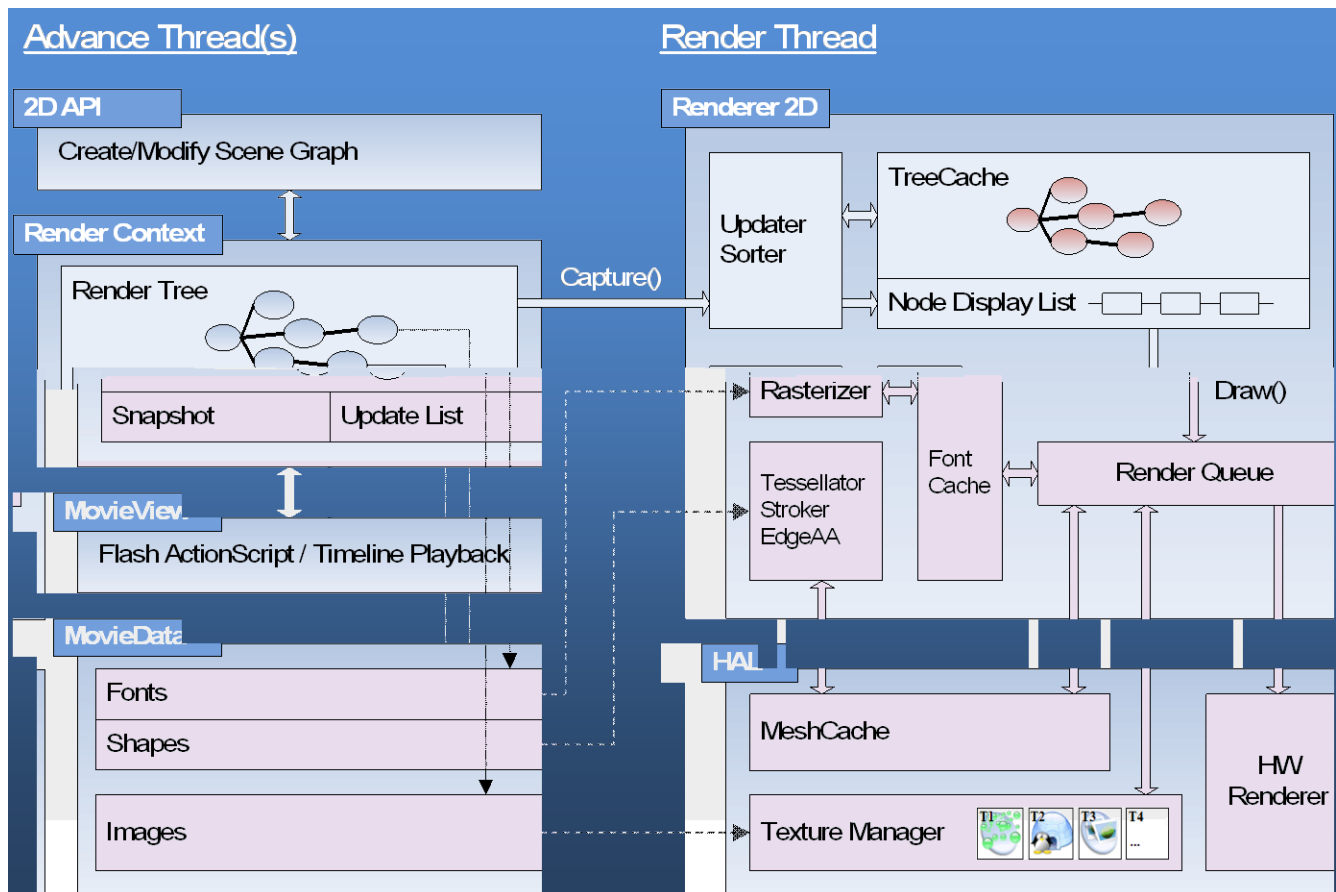


3.1



4 렌더러 설계:

Scaleform 4.0



1:

MovieView MovieData Scaleform

. Render Context Render Tree

2D

. Scaleform

Movie

```

graph LR
    subgraph "2D Renderer"
        RQ[Render Queue]
        HA[HAL APIs]
    end
    subgraph "HAL"
        CPU[CPU]
        AI[AI]
    end
    subgraph "Display"
        D[Display]
    end
    RQ --> AI
    AI --> CPU
    CPU --> D
  
```

The diagram illustrates the rendering pipeline. It starts with a 2D Renderer, which contains a Render Queue and HAL APIs. The Render Queue sends data to the AI (Application Interface) component within the HAL layer. The AI component then sends data to the CPU, which finally sends data to the Display component.

5 사용자 정의 데이터로 렌더링

Scaleform

.

.

5.1

AS2

"renderString" "renderFloat" AS3

"setRenderString"

"setRenderFloat" ActionScript

. AS2/AS3

. Movie clip / float

UserDataState::Data the HAL::PushUserData

Render::HAL UserDataStack

.

.

movie clip / Float

. Movie clip

UserDataState::Data HAL::PopUserData

.

5.2

movie clip

HAL

. HAL

2D

'm' movie clip

AS3

AS3 SWF

.

```
import scaleform.gfx.*;
DisplayObjectEx.setRenderString(m, "Abc");
DisplayObjectEx.setRenderFloat(m, 17.1717);
```

4.1

Src/Render/ShaderData.xml

```
<ShaderFeature id="Position">
  <ShaderFeatureFlavor id="Position2d" hide="true"/>
  <ShaderFeatureFlavor id="Position3d"/>
  <ShaderFeatureFlavor id="Position2dOffset"/>
</ShaderFeature>
```

"Position2dOffset"

snippet

"Position2dOffset" snippet

```
<ShaderSource id="Position2dOffset" pipeline="Vertex">
  Position2d(
    attribute float4 pos      : POSITION,
    varying   float4 vpos     : POSITION,
    uniform   float4 mvp[2],
    uniform   float  poffset)
  {
    vpos = float4(0,0,0,1);
    vpos.x = dot(pos, mvp[0]) + poffset;
    vpos.y = dot(pos, mvp[1]) + poffset;
  }
</ShaderSource>
```

snippet x y 'poffset' (uniform)

"Position2d" snippet

D3D9_Shader.cpp ShaderInterface::SetStaticShader

```
// See if we have user data on the stack.
if (pHal->UserDataStack.GetSize() > 0 )
{
  const UserDataState::Data* data = pHal->UserDataStack.Back();
  if (data->Flags &
(UserDataState::Data::Data_Float|UserDataState::Data::Data_String) &&
    data->RendererString.CompareNoCase("abc") == 0)
```

```

    {
        // Modify the shader we use, if we find the matching user data we were
expecting.
        vshader = (VertexShaderDesc::ShaderType)((int)vshader +
VertexShaderDesc::VS_base_Position2dOffset);
        shader = (FragShaderDesc::ShaderType) ((int)shader +
FragShaderDesc::FS_base_Position2dOffset);
    }
}

```

```

HAL UserDataStack ("abc" float )
. Position2dOffset
. ShaderData.xml
D3D9_ShaderDescs.h .

```

```

'poffset' (uniform)

```

```

. ShaderInterface::Finish .

```

```

// See if we have user data on the stack.
if (pHal->UserDataStack.GetSize() > 0 )
{
    const UserDataState::Data* data = pHal->UserDataStack.Back();
    if (data->Flags &
(UserDataState::Data::Data_Float|UserDataState::Data::Data_String) &&
    data->RendererString.CompareNoCase("abc") == 0)
    {
        // Add the poffset uniform into the uniform data.
        for ( unsigned m = 0; m < Alg::Max<unsigned>(1, meshCount); ++m)
        {
            float poffset = data->RendererFloat / 256.0f;
            SetUniform(CurShaders, Uniform::SU_poffset, &poffset, 1, 0, m);
        }
    }
}

```

```

'meshCount' meshCount
draw( ) meshCount . HAL
movie clip 'm' .

```

5.3

HAL Scaleform

Scaleform

Scaleform

/

AS2 "disableBatching"

AS3 "disableBatching"

. AS2/AS3

6 GFxShaderMaker

6.1

```
Scaleform 4.1      GfxShaderMaker    Scaleform
                  (Render_D3D9)          ShaderData.xml
                  .                      ,
                  .                      .
                  .                      D3D9
```

```
Src/Render/D3D9_ShaderDescs.h
Src/Render/D3D9_ShaderDescs.cpp
Src/Render/D3D9_ShaderBinary.cpp
```

6.2 API HAL

6.2.1 D3D9

```
D3D9 HAL ShaderModel 2.0 ShaderModel 3.0
HAL
GFXShaderMaker '-shadermodel'
```

```
GfShaderMaker.exe -platform D3D1x -shadermodel SM30
```

ShaderModel 2.0

. ShaderModel 2.0

InitHAL

, .

6.2.2 D3D1x

D3D1x HAL

3

. (D3D11 D3D10.1)

FEATURE_LEVEL_10_0

FEATURE_LEVEL_9_3

FEATURE_LEVEL_9_1

, HAL .

. GfxShaderMaker '-featurelevel' ,

. :

GfxShaderMaker.exe -platform D3D1x -featurelevel

FEATURE_LEVEL_10_0,FEATURE_LEVEL_9_1

FEATURE_LEVEL9_3

. FEATURE_LEVEL_9_3

InitHAL

, (FEATURE_LEVEL_9_1)

.